



YOUR PROJECTS

PaperRAG and SchemaMind
pictures, pipelines and Q&A

Your Projects, Explained

What happens inside **PaperRAG** and **SchemaMind**, from the moment a question is typed to the moment an answer comes back: every step drawn as a picture, explained in plain words, tied to the file in your code, and followed by the how, why, what-if and why-not questions interviewers ask.

the two ideas to remember

```
# PaperRAG: refuse when nothing is close enough
abstain = top_score < threshold          # 0.50, from the eval

# SchemaMind: exactly one read query, or nothing
if len(statements) != 1:
    return ValidationResult(False, ...)
```

PDF blocks

Reading order

Chunks & overlap

Embeddings

FAISS

The "I don't know" guard

Citations

FastAPI

Streamlit

Evaluation

Text-to-SQL

Schema cards

Templates

sqlglot

Read-only

Time limit

Repair loop

Semantic layer

Tests

Written for a beginner. Describes the code in github.com/sudhanshusharmasikar-ctrl/paperrag and [.../schemamind](https://github.com/sudhanshusharmasikar-ctrl/schemamind) after build sessions 1–11. Outputs marked "real" were produced by running that code; scores marked "example" are illustrations.

How to Read This Guide

This guide explains your two main projects, **PaperRAG** and **SchemaMind**, from zero. Every step has a picture, the idea in plain words, and the file in your code where it happens.

- **Read each part in order:** the big picture first, then the steps, then the questions.
- **Words in bold** are explained where they first appear and collected in the glossary (3.4).
- **It describes the code as it is now**, after build sessions 1–11. Your GenAI notes (Ch 17–18) are older and still list some fixed bugs as open; where they differ, trust this guide.
- Outputs marked **real** come from running your code; scores marked **example** are only illustrations.

✓ How to study with it

1. Look at the big picture (1.2 and 2.3) and say out loud what each numbered box does.
2. Read the steps with the code open next to you.
3. Cover the answers in the question sections, answer out loud, and mark the hard ones.

Contents

PART 1	PaperRAG: Ask Your Research Papers	3
1.1	The problem, in plain words	3
1.2	The whole pipeline in one picture	4
1.3	Building the library, step by step	4
1.4	Answering a question, step by step	8
1.5	How the parts talk to each other	11
1.6	One question, from start to finish	12
1.7	How it is measured and tested	12
1.8	What we changed, and what we tried	14
1.9	Questions and answers	16
PART 2	SchemaMind: Ask a Database in Plain English	22
2.1	The problem, in plain words	22
2.2	The sample shop database	23
2.3	The whole pipeline in one picture	24
2.4	Each step, in order	24
2.5	One question, from start to finish (real output)	29
2.6	How it is measured and tested	30
2.7	What we changed, and what we tried	32
2.8	Questions and answers	33
PART 3	Seeing Them Together	39
3.1	The same skeleton	39
3.2	Explaining each project in one minute	40
3.3	Questions that cover both projects	41
3.4	Glossary	43

PART 1 Retrieval-augmented generation · FAISS · FastAPI

AI

PaperRAG: Ask Your Research Papers

IN THIS PART

- What problem PaperRAG solves, in plain words
- **The whole pipeline in one picture**
- Building the library: blocks, reading order, chunks, embeddings, the index
- Answering: **hybrid search** (meaning + keywords), the **"I don't know" guard**, the answer
- How the web page and the server talk to each other
- One question, followed from start to finish
- How it is measured and tested, and what we fixed
- **Questions and answers:** how, why, what if, what more, why not, what we tried

1.1 The problem, in plain words

Imagine 40 research papers on your laptop. You want to ask *"Which optimizer did the transformer paper use?"* and get an answer you can **trust**, together with **where** it came from, so you can open the paper and check.

Why not just ask ChatGPT?

- It may never have read your papers: new ones, private ones, your own drafts.
- It can **make things up** in a confident voice. This is called **hallucination**.
- It doesn't tell you the file and page it used.



An open-book exam

A normal LLM is a student answering from memory. PaperRAG turns it into an **open-book exam**: first find the right pages in your own books, then answer **only** from those pages, and write down which page you used. If the books don't cover the question, the honest student says *"I don't know"*.

That idea is called **RAG**, retrieval-augmented generation. *Retrieval* means finding the right pieces of text; *augmented generation* means writing the answer from those pieces. PaperRAG adds two things to basic RAG: **page-level citations**, and a **guard** that refuses when your papers don't contain the answer.

1.2 The whole pipeline in one picture

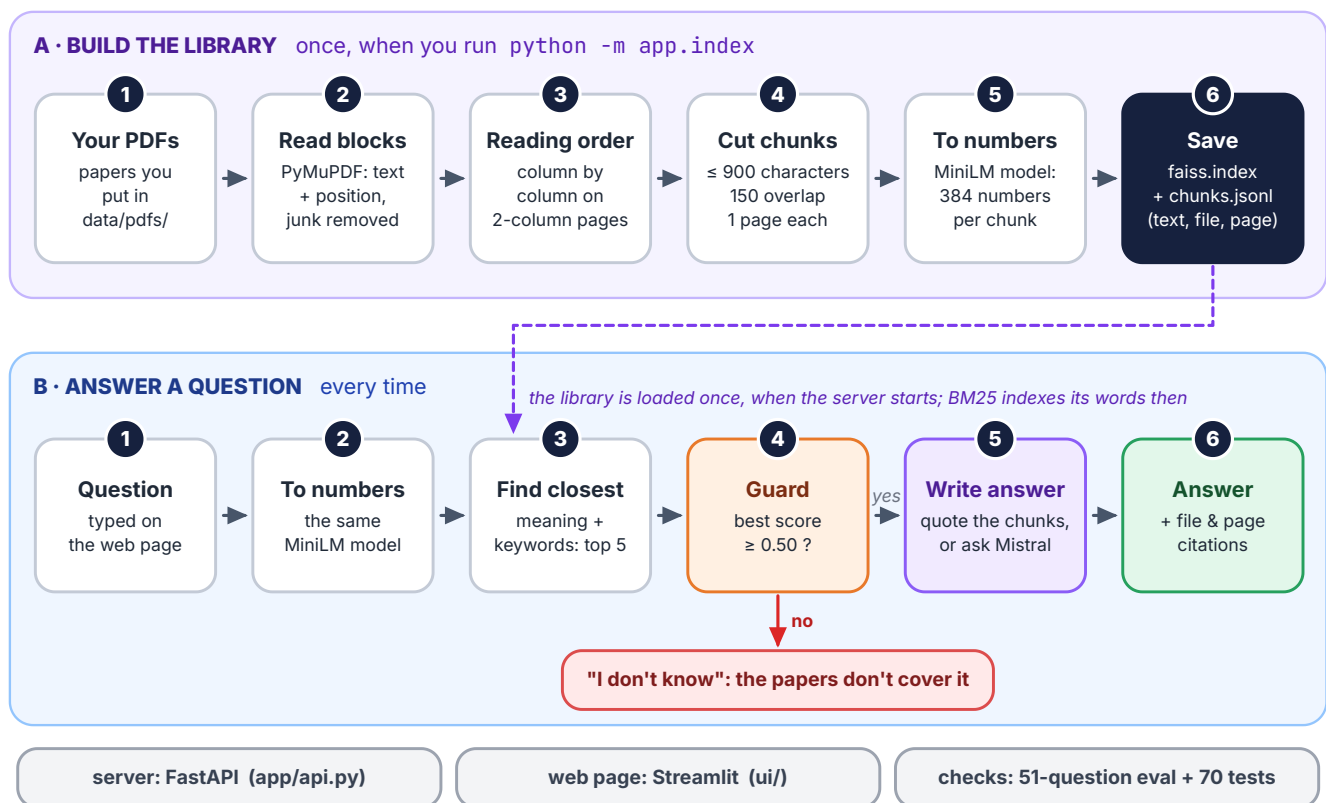


Figure 1.1 PaperRAG has two phases. **A** runs once and builds a searchable library from your PDFs. **B** runs for every question: find the closest chunks, check they are close enough, then answer with citations.

Read the picture as two rows:

- **Row A, building the library**, happens **once**, when you run `python -m app.index`. It reads every PDF, cuts the text into small pieces called **chunks**, turns each chunk into **384 numbers** that describe its meaning, and saves everything in the `storage/` folder.
- **Row B, answering**, happens **every time** someone asks. The question is turned into 384 numbers by the same model, the closest chunks are found by meaning and by keywords, a **guard** checks whether the best one is close enough, and the answer comes back with the file and page of every passage.

The rest of this part goes through the numbered boxes one by one.

1.3 Building the library, step by step

A1 · Your PDFs

You copy papers into `data/pdfs/` and run `python -m app.index`, which calls `build()` in `app/index.py`. It goes through the PDFs one by one. A PDF that gives no text at all, usually a scanned paper that is really a picture of text, is skipped and listed at the end.

A2 · Read the text in blocks

A PDF doesn't store paragraphs: it stores letters at positions on a page. **PyMuPDF** (the `pymupdf` library) groups letters into **blocks**, roughly one visual paragraph each, together with the block's position on the page `(x0, y0, x1, y1)`. Two clean-up steps follow, both in `app/chunking.py`:

Step	What it does	Example
<code>clean()</code>	fixes PDF oddities	the one-character ligature "fi" becomes "fi"; "multi-" at a line end followed by "modal" becomes "multimodal"
<code>_is_noise()</code>	throws away blocks that aren't prose	page numbers ("12"), short labels ("Table 3"), rows of numbers from tables

Why bother? Junk text gets turned into numbers too, and it makes the search worse.

A3 · Put the blocks in reading order

On a two-column paper, sorting blocks from top to bottom mixes the columns: left paragraph, right paragraph, left, right. PaperRAG reads **column by column** instead, in `_reading_order()`:

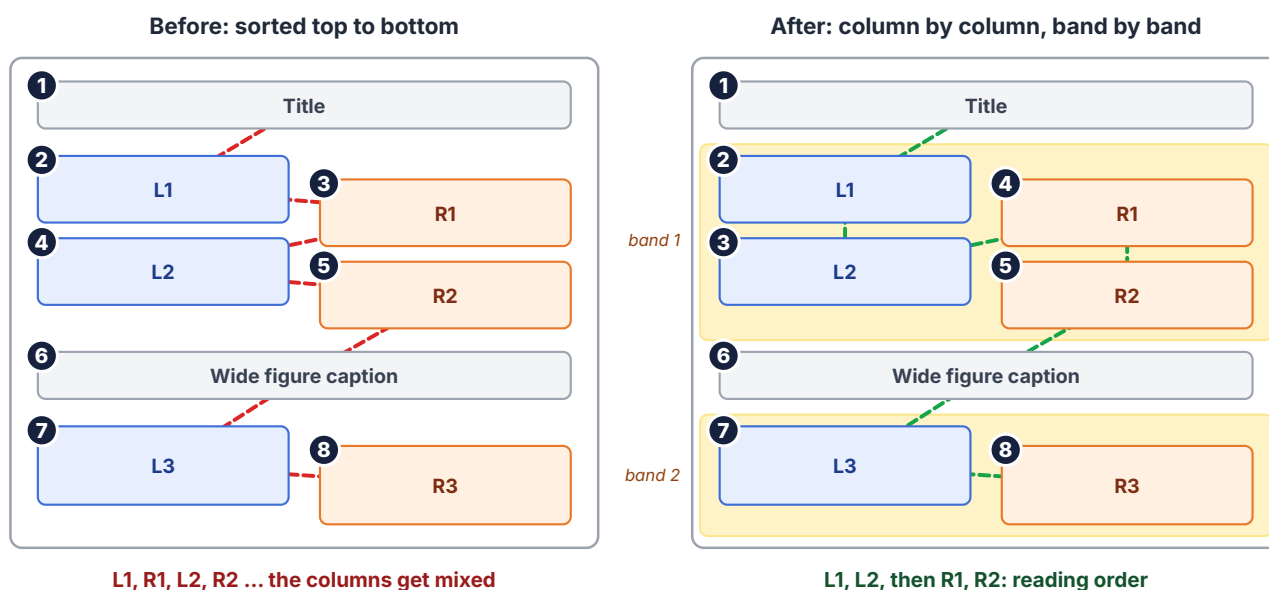


Figure 1.2 Left: the old order mixed the columns. Right: blocks that cross the middle of the page (the title, a wide figure caption) cut the page into **bands**; inside each band the left column is read first, then the right one.

How it decides: a block that **crosses the middle** of the page is **wide**: a title, an abstract, a wide caption. Every other block sits in the left or the right column. Wide blocks cut the page into bands, and each band is read left column first, then right column. On a one-column page almost every block crosses the middle, so nothing changes. This was fixed in session 3 (see 1.8).

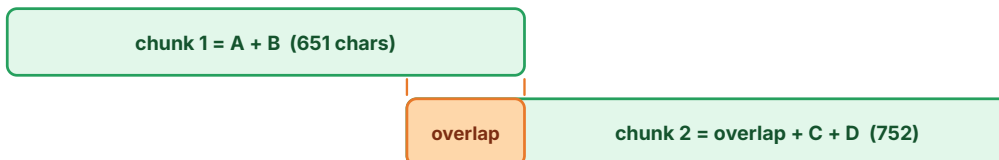
A4 · Cut the text into chunks

A whole paper is far too long to search as one piece, and the embedding model reads only about 1,000 characters at a time. So the blocks of each page are packed into **chunks** by `_pack()`:

Page 1: blocks in reading order

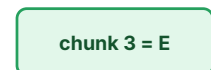


Chunks: whole blocks packed up to 900 characters



The last 150 characters of chunk 1 are repeated at the start of chunk 2, so a sentence cut at the edge is complete in one of them. A new page always starts a new chunk.

Page 2



never mixed
with page 1:
1 chunk =
1 page

Figure 1.3 Whole blocks are packed into chunks of at most 900 characters. The end of one chunk is repeated at the start of the next (the overlap), and a chunk never contains text from two pages.

Rule	Value	Why
Maximum size	900 characters	the model reads only about 256 word pieces ($\approx$ 1,000 characters); anything longer would be cut off without a warning
Overlap	150 characters	a sentence that falls on the edge is still complete in one of the two chunks; the overlap shrinks when the next paragraph is long, so the 900 limit holds
One page per chunk	always	every chunk has exactly one page number , which is what makes page citations possible

Chunks shorter than 120 characters are dropped (usually headers or stray labels). A single paragraph longer than 900 characters is split at sentence ends. Each chunk becomes one line of `storage/chunks.jsonl`; this one is **real**, made by the current code from a small test PDF (the text is shortened here):

JSON one line of `storage/chunks.jsonl`, spread over several lines

```
{
  "chunk_id": "transformers::p2::c0",
  "doc_id": "transformers",
  "source": "transformers.pdf",
  "page": 2,
  "text": "We train with the Adam optimizer, a learning rate warmup of ..."
```

Read the `chunk_id` as: file `transformers`, page 2, chunk 0 of that page.

A5 · Turn every chunk into numbers

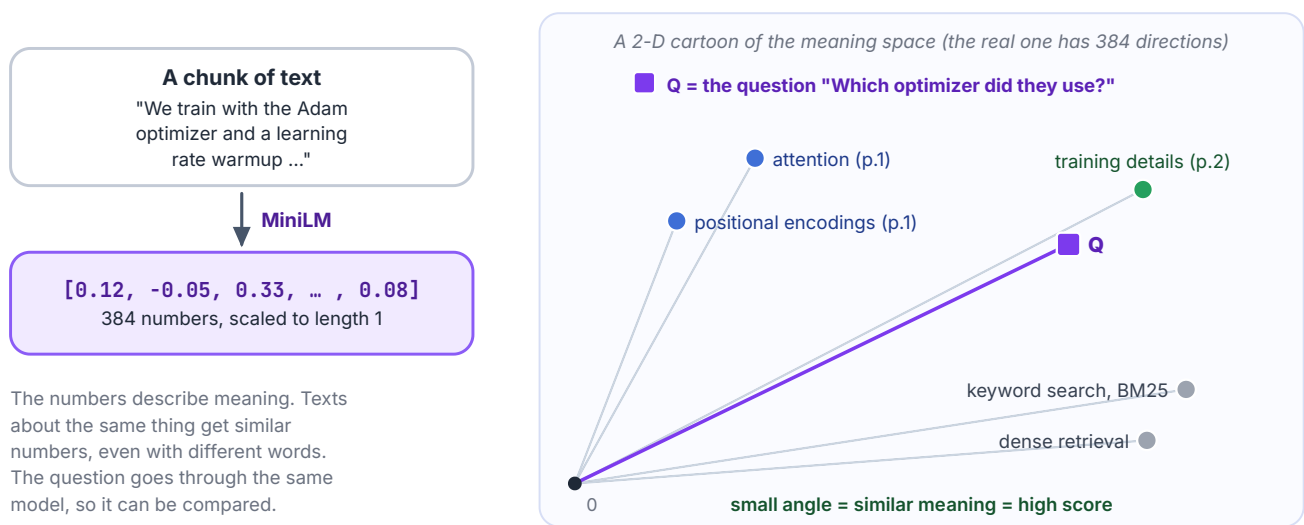


Figure 1.4 An embedding model turns text into a list of numbers. Texts with similar meaning get numbers that point in a similar direction, so the question lands next to the chunk that answers it.

An **embedding model** reads a piece of text and outputs a list of numbers, called a **vector**, that describes its meaning. PaperRAG uses **all-MiniLM-L6-v2** ("MiniLM"): every chunk becomes **384 numbers**. Texts about the same idea get similar numbers even when they use different words.

Every vector is **normalised**, which means scaled to length 1. Comparing two such vectors is then one multiplication, the **dot product**, and its result is the **cosine similarity**: close to 1 means the same meaning, close to 0 means unrelated.

Why MiniLM: it is small (about 80 MB), fast on a laptop's processor, free, and works offline. A bigger model might match meaning a little better, but this project has to run on a free server.

A6 · Save the library

File	What is in it	Why
<code>storage/faiss.index</code>	the 384 numbers of every chunk	FAISS is a library built for searching vectors fast; it stores only numbers
<code>storage/chunks.jsonl</code>	the text, file name and page of every chunk, one per line	so a search result ("row 17") can be turned back into text and a page

Row 17 in the index is line 17 in the file. When the server loads them, `Load()` checks that the counts match, and refuses to start if they don't ("out of sync — rebuild").

PaperRAG uses the index type **IndexFlatIP**. *Flat* means the question is compared with **every** chunk, an exact search; *IP* means inner product, which is the dot product above.

1.4 Answering a question, step by step

B1 · The question arrives

You type into the Streamlit page, which sends the question to the server's `/ask` endpoint as JSON (a simple text format for data):

JSON what the web page sends to POST `/ask`

```
{"question": "Which optimizer did they use?", "top_k": 5, "threshold": 0.50, "mode": "extractive"}
```

The server checks the input first: 3 to 1,000 characters, `top_k` between 1 and 20, `mode` either `extractive` or `mistral`. Anything else is rejected with an error (HTTP 422) before any work starts.

B2 · The question becomes numbers

The question goes through the **same** MiniLM model and becomes 384 numbers. It has to be the same model: numbers from two different models are like coordinates from two different maps.

B3 · Find the closest chunks: meaning and keywords

Two searches run, and their rankings are merged (session 10):

- **By meaning.** FAISS compares the question's vector with every chunk's vector: cosine similarity. With a few hundred papers that is tens of thousands of comparisons, which takes milliseconds. This finds a passage that says the same thing in other words.
- **By keywords.** BM25 (`app/bm25.py`) scores each chunk by the question's words it contains. A rare word like "WordPiece" counts far more than a common one like "model", repeating a word helps less and less, and a long chunk doesn't win just by holding more words. This finds the exact terms that meaning search can miss.

The two lists are merged with **reciprocal rank fusion (RRF)**: each list gives a chunk $1 / (60 + \text{its rank})$, and the two shares add up. Only ranks are used, so a cosine score is never compared with a BM25 score. The 5 chunks with the highest total are cited.

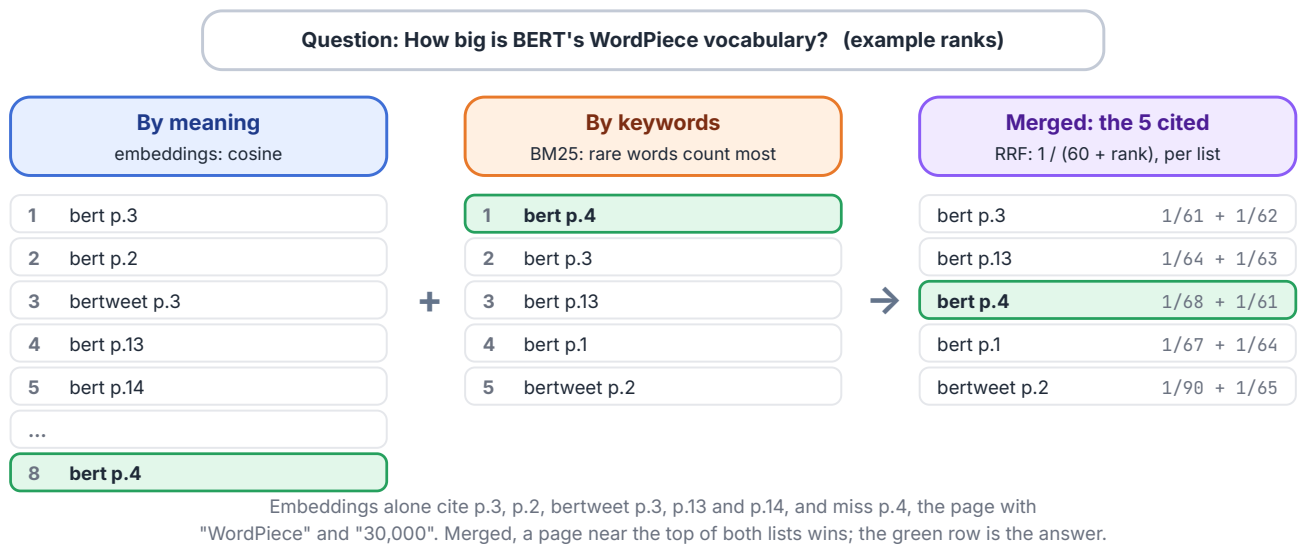


Figure 1.5 How the two rankings merge (example ranks). Meaning alone ranks the page with the answer 8th, below the 5 that get cited; keywords rank it 1st. Merged, it makes the 5.

Last, each page is cited **at most once** (session 11). A page is often cut into several chunks that rank close together; keeping only each page's best chunk makes the 5 citations 5 different pages. It can't drop a page that was already cited, so the first citation stays the same and the right page can only be gained.

Your evaluation measured both steps: the right page was among the citations for 19 of the 30 answered questions with meaning alone, 24 with hybrid search, and 26 with one chunk per page (1.7).

`PAPERRAG_RETRIEVAL=dense` and `PAPERRAG_DISTINCT_PAGES=0` switch them off.

B4 · The guard

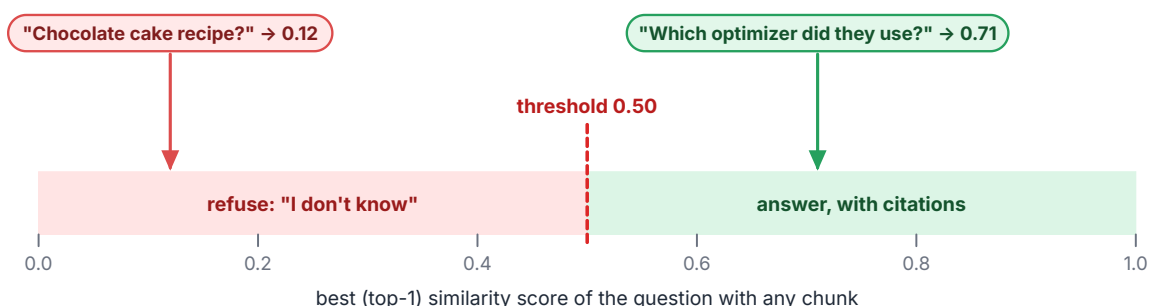


Figure 1.6 The guard compares only the best score with the threshold. Below 0.50 PaperRAG refuses; at 0.50 or above it answers.

If even the best chunk scores below the **threshold**, your papers probably don't cover the question, so PaperRAG refuses instead of guessing. The refusal shows both numbers (the format is **real**; the score is an example):

OUTPUT the refusal message, wrapped onto two lines here

I don't have enough supporting material in the indexed papers to answer that. The closest passage scored 0.123, below the 0.500 threshold.

Two details matter:

- It uses the **best (top-1)** score, not the average of the top 5. The average drops as you ask for more results, so the cut-off would quietly depend on `top_k`.
- The threshold is **0.50**, chosen in session 7 from your own evaluation (1.7). It started at 0.35, which let 10 of the 17 unanswerable test questions through.
- With hybrid search the guard still uses the **best cosine score**, not the merged ranks. So it refuses exactly the same questions as before; hybrid search changes only which pages are cited.

B5 · Write the answer

Mode	How the answer is made	Good	Not so good
<code>extractive</code> (default)	the passages are returned word for word, numbered, with file, page and score	invents nothing; no API key; free	you read passages, not a short answer
<code>mistral</code>	the passages and the question go to the Mistral LLM with strict rules: use only the passages, cite them as [1], [2], and reply <code>INSUFFICIENT_CONTEXT</code> if they don't contain the answer	a short, readable answer	needs an API key; the model could still slip

In mistral mode there is a **second guard**: if the model replies `INSUFFICIENT_CONTEXT` (the passages were on topic but didn't contain the answer), PaperRAG turns that into a refusal too. The **temperature** is 0, so the same question gives the same answer.

B6 · The reply

JSON what /ask sends back (the shape is real, the values are an example)

```
{
  "question": "Which optimizer did they use?",
  "answer": "Extractive mode -- passages returned verbatim ...",
  "abstained": false, "top_score": 0.712, "threshold": 0.50,
  "mode": "extractive",
  "citations": [
    {
      "n": 1, "source": "transformers.pdf", "page": 2,
      "score": 0.712, "chunk_id": "transformers::p2::c0"
    }
  ],
  "latency_ms": 41.3
}
```

The citations come from the chunks' saved file and page, **not** from the LLM, so the model can't invent a page number. Each one shows its chunk's cosine score; in hybrid search the order is the merged order, so a lower score can come before a higher one.

1.5 How the parts talk to each other

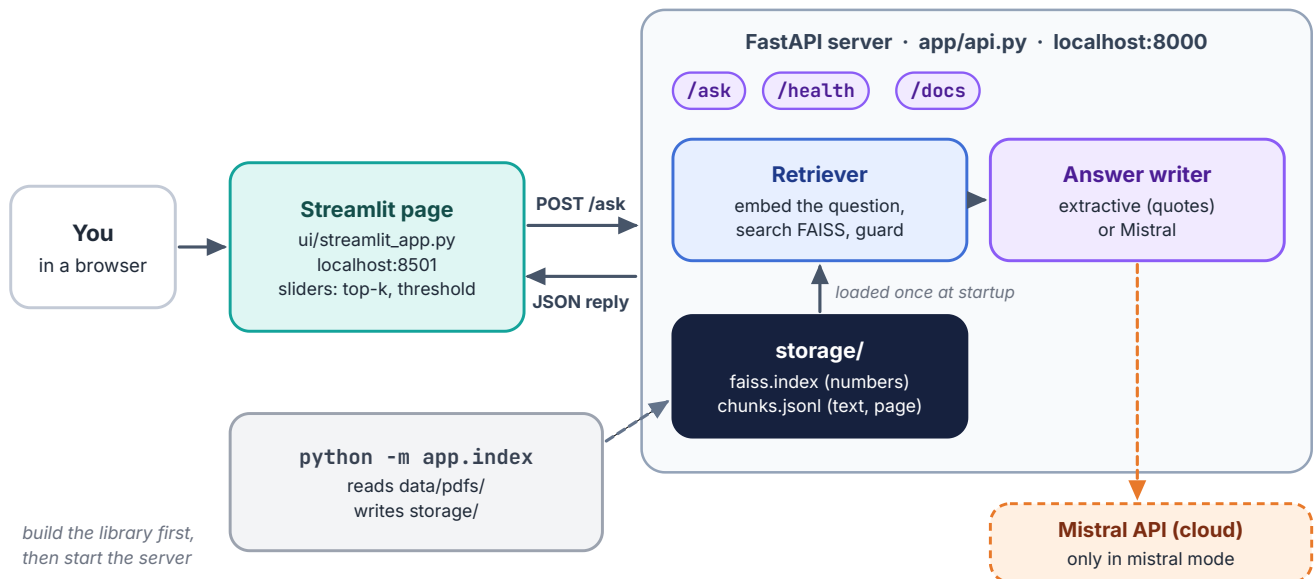


Figure 1.7 You use the Streamlit page; the page talks to the FastAPI server over HTTP; the server keeps the model and the library in memory and calls Mistral only in mistral mode. The library is built beforehand by the indexing script.

- **The FastAPI server** (`app/api.py`) is a Python web server. It loads the model and the library **once**, when it starts (the `lifespan` function), then answers requests at its endpoints: `/ask` for questions, `/health` to check that the library is loaded and how many chunks it has, and `/docs` , an automatic page where you can try the API in the browser.
- **The Streamlit page** (`ui/streamlit_app.py`) is what you use. It contains no search code: it sends HTTP requests to the server and shows the reply. Its sliders change `top_k` , the threshold and the mode.
- **Why two programs?** The page proves the API works on its own, and any other program, a mobile app or a script, could use the same API.

You have already seen `/health` working: `{"status":"index_missing","chunks":0,...}` means the server runs but no library has been built yet.

1.6 One question, from start to finish

Question: “Which optimizer did the transformer paper use?” The scores are an **example**.

#	What happens	Where in the code	Result
1	The page sends the question to <code>/ask</code>	<code>ui/streamlit_app.py</code>	a JSON request
2	The input is checked (length, <code>top_k</code> , mode)	<code>AskRequest</code> in <code>app/api.py</code>	OK
3	The question becomes 384 numbers	<code>embed()</code> in <code>app/index.py</code>	one vector
4	FAISS and BM25 rank the chunks; RRF merges them; the top 5 pages, one chunk each	<code>Retriever.__call__</code> in <code>app/retrieve.py</code>	first: <code>transformers.pdf p.2</code> ; best score 0.71
5	The guard compares 0.71 with 0.50	the same function	answer allowed
6	The extractive answer is built	<code>answer()</code> in <code>app/generate.py</code>	passages starting <code>[1 transformers.pdf p.2</code>
7	The reply goes back with citations and timing	<code>ask()</code> in <code>app/api.py</code>	shown on the page

Now ask “What is a good chocolate cake recipe?”. Step 4 finds nothing close (best score perhaps 0.12), the guard stops the question at step 5, and the page shows the refusal message with both numbers.

1.7 How it is measured and tested

Tests check that the code does what it promises. There are 70, run with `pytest`, offline, in about three seconds. They don’t download MiniLM: a tiny stand-in that counts words replaces it, so the tests are fast and give the same result every time. They cover cleaning, reading order, chunk sizes and overlap, the saved files, the guard, BM25 and hybrid search, one chunk per page, the answer format, the API, the evaluation’s arithmetic, the question checker, settings from `.env` and the web page’s slider. GitHub runs them after every push, on Python 3.11 and 3.14 (see 3.3).

Evaluation measures how *good* the answers are, which tests can’t do. In session 7 you built the question set, `eval/questions.jsonl`: **51 questions** about your 8 papers (905 chunks).

- **34 answerable**, each with the page that answers it and a short **evidence phrase** copied from that page.
- **17 unanswerable**: on topic, but about models none of your papers mention (LLaVA, Whisper, T5 ...).

`python -m eval.check_questions` confirms that every evidence phrase is really on its page and that no paper names the models in the unanswerable questions. Then `python -m eval.run_eval` asks every question and counts (**real**, at threshold 0.50):

Number	Meaning	Your result
Unsupported answers, no guard	answers with no support in the papers, if nothing ever refused	33.3%: all 17 unanswerable get an answer
Unsupported answers, with guard	the same, with the guard on	5.9%: only 3 of 51
False refusals	answerable questions that were refused	11.8%: 4 of 34
Citation hit rate	answered questions whose right page is among the citations	86.7%: 26 of 30 (80.0% with hybrid search alone, 63.3% by meaning alone)
Retrieval time	turning the question into numbers and searching	median 11.6 ms, p95 12.5 ms

`--sweep` repeats this for every threshold from 0.20 to 0.66. Your real curve:



Figure 1.8 Your sweep (real). Raising the threshold makes more unanswerable questions refused, but from 0.44 on also more good ones. At 0.50 the blue line has dropped to 3 of 17 while the orange one is still at 4 of 34.

Why 0.50. Up to 0.42 no good question is refused, but 8 or more of the 17 unanswerable ones still get an answer. From 0.42 to 0.50, five more unanswerable questions are refused for the price of four good ones. Above 0.50 it gets expensive: each further refusal of an unanswerable question costs one to three good ones, and 0.56 refuses a third of the good questions.

What went wrong at 0.50 (`--threshold 0.50` lists every miss):

- **3 unanswerable questions got an answer**, all just above the threshold (0.507 to 0.549): the size of LAION-5B, T5's training data and XLNet's objective. Your papers don't mention these models, but they do talk about training data and objectives, so a passage on the same topic scores high. A

score threshold catches *off-topic* questions; it can't catch an *on-topic* question whose answer is missing. That's the job of mistral mode's second guard.

- **4 good questions were refused** (0.434 to 0.478). Each asks for one small detail: InfoNCE in CLIP, the fusion methods compared with TMPT, the tokenizer MLLM-SD uses, where ViT puts LayerNorm. Such narrow questions score lower than broad ones.
- **4 answers cite the wrong page** (6 with hybrid search alone, 11 by meaning alone). Each now cites five different pages, but the page with the answer ranks below them: the hardware BERT-Large was trained on, the name of the new multi-turn stance dataset, the model that writes MLLM-SD's captions, and how ViT's classification head changes for fine-tuning.

Three setups compared (sessions 10 and 11). `--threshold 0.50` runs every question three times, each setup adding one change. The guard is the same, so the same 30 questions are answered (**real**):

Of the 30 answered questions	Meaning alone	Hybrid	Hybrid, 1 chunk per page
Right page among the 5 citations	19 (63%)	24 (80%)	26 (87%)
Right page cited first	13	12	12
Different pages cited (average)	4.0	4.0	5.0

- **Hybrid search: 6 more, 1 lost.** It helped most with an exact term: the Transformer paper's other name for self-attention ("intra-attention"), the size of BERT's **WordPiece** vocabulary, ViT's position embeddings. It lost T-MAD's encoders.
- **One chunk per page: 2 more, none lost.** In 5 of the 6 misses left after hybrid search, one page had taken two or three of the five slots (three chunks of T-MAD's page 15, for one). Freeing them brought in the right page for T-MAD's encoders, the question hybrid had lost, and for when a self-attention layer is cheaper than a recurrent one.
- **The first citation got no better**, 13, then 12 and 12: both changes widen what makes the five, while putting the best one first is a reranker's job.
- One caveat: the questions were drafted from the papers' own passages, so they share words with the right page, which favours keyword search.

1.8 What we changed, and what we tried

Session	Change	Why it matters
1	Every library fixed to a tested version	the project installs the same way everywhere, including Python 3.14 on your Mac
2	34 tests with a stand-in embedder	writing them uncovered two bugs, fixed in session 3
3	Two-column pages read column by column	chunks no longer mix sentences from two columns
3	The overlap can't push a chunk past 900 characters	no chunk is longer than the model can read, except a single sentence over 900 characters
3	An overlap of 0 no longer repeats whole chunks	in Python, <code>text[-0:]</code> is the whole string, so chunks had snowballed to 1,877 characters

Session	Change	Why it matters
6	The tests run on GitHub after every push (CI)	a change that breaks something shows a red cross before it is merged
7	51 evaluation questions, a checker for them, real numbers in the README	the results table was empty; now every number comes from your papers
7	Threshold 0.50 instead of 0.35	0.35 let 10 of the 17 unanswerable questions through
7	<code>.env</code> is really read	a threshold or API key set there was silently ignored
7	The web page's slider starts at the server's threshold	its own 0.35 would have overridden the new default
10	Hybrid search: BM25 keywords merged with the embeddings by RRF	the right page is cited for 24 of 30 answered questions instead of 19
10	The evaluation runs both searches on the same questions	the guard is the same in both, so only the cited pages can differ
11	Each page is cited at most once (its best chunk)	the right page for 26 of 30 answered questions instead of 24, none lost
11	The evaluation compares three setups, one change at a time	each column shows what one change did

What we tried for the two-column fix. The first idea, from your notes, gave every block a single sort key: wide blocks first, then the left column, then the right. A test showed it breaks normal pages: a short line on a one-column page moved to the end, and a wide caption in the middle jumped to the top. The band method passed all three layout tests and read a realistic five-page two-column paper in the right order, while one-column documents came out exactly as before.

What we tried in session 7. The first speed measurement gave a median of 9 ms but a p95 of 164 ms. On a Mac the model runs on the GPU ("mps"), which most likely does one-off setup work the first few times it sees a question. Switching to the CPU made both commands crash with a *segmentation fault*, most likely because faiss and PyTorch each bring their own copy of the OpenMP threading library. So the GPU stayed, and questions are timed only after one untimed warm-up pass: p95 10.4 ms.

What we tried in session 10. BM25 is about 40 lines in `app/bm25.py`, so it needs no new library and every line is tested. The file was first called `keyword.py`, which would hide Python's own `keyword` module whenever a script inside `app/` was run directly, so it became `bm25.py`. Hybrid search went in switched off and became the default only after your evaluation showed 19 → 24. One chunk per page (session 11) went the same way, 24 → 26; a test checks on 200 random rankings that it never drops a page the plain ranking cited.

Still missing

The first citation is right only 40% of the time (12 of 30), and 4 right pages still rank below five others; a reranker would target both. 3 of 17 unanswerable questions still get an answer, because a score guard can't spot an on-topic question whose answer is missing. Mistral mode hasn't been run with your key yet: it needs a model your plan allows. Tables and scanned pages are skipped.

1.9 Questions and answers

HOW DOES IT WORK?

Q1 How does PaperRAG know which page an answer came from?

Every chunk is built from one page only, and its page number is saved next to its text in `chunks.jsonl`. When a chunk is found, its page comes with it. The LLM never chooses the page, so it can't invent one.

Q2 How does it decide to say "I don't know"?

It compares the best similarity score between the question and any chunk with a threshold, 0.50, which the evaluation chose. Below it, the papers probably don't cover the question, so it refuses. In mistral mode a second check also refuses when the model replies `INSUFFICIENT_CONTEXT`.

Q3 How does it read a two-column paper in the right order?

Blocks that cross the middle of the page (title, abstract, wide captions) cut the page into bands. Inside each band it reads the left column from top to bottom, then the right column. One-column pages are unchanged, because almost every block there crosses the middle.

Q4 How is text turned into numbers, and how are they compared?

MiniLM turns each chunk, and later the question, into 384 numbers scaled to length 1. The dot product of two such vectors is their cosine similarity: close to 1 means similar meaning. FAISS computes it for every chunk and keeps the top 5.

Q5 How does hybrid search work?

Two rankings: FAISS by meaning (cosine similarity) and BM25 by keywords, where rare words count most. Reciprocal rank fusion merges them: each list gives a chunk $1 / (60 + \text{rank})$, and the shares add up. Then each page keeps only its best chunk, the top 5 are cited, and the guard still checks the best cosine score.

Q6 How do the web page and the server work together?

The Streamlit page sends the question to the FastAPI server as JSON, with an HTTP POST to `/ask`. The server, which loaded the model and the library once at startup, runs the search, the guard and the answer step, and sends JSON back for the page to display.

WHY WAS IT BUILT THIS WAY?

Q7 Why cut papers into chunks instead of searching whole papers?

A whole paper's meaning blurs into one vector, and the model reads only about 1,000 characters anyway. Small chunks give precise matches and precise page citations.

Q8 Why 900 characters with 150 overlap?

900 stays under what MiniLM can read (about 256 word pieces), so nothing is silently cut off. The overlap keeps a sentence that falls on a chunk edge complete in one of the chunks. Both are starting values that the evaluation can compare against others.

Q9 Why exact search (FAISS IndexFlatIP) and not HNSW?

A few hundred papers give tens of thousands of chunks, and checking all of them takes milliseconds with perfect accuracy. HNSW is approximate and has settings to tune; it solves a speed problem this project doesn't have yet.

Q10 Why MiniLM and not a bigger embedding model?

It's small (about 80 MB), fast on a laptop, free, and works offline, so the project runs on a free server. A bigger model might find slightly better matches, and the evaluation would show whether that's worth the cost.

Q11 Why is extractive mode the default?

It needs no API key, costs nothing, and can't invent anything, because it only quotes. It's also a fair baseline to compare the LLM mode against.

Q12 Why use the best score, not the average of the top 5?

The average drops as you ask for more results, so the cut-off would quietly change whenever `top_k` changes. The best score doesn't depend on `top_k`.

Q13 Why merge ranks (RRF) instead of adding the scores?

A cosine lies between -1 and 1; a BM25 score can be 0 or 25, depending on the words. Adding them would need a weight to tune. Ranks are comparable as they are, and 60, the usual constant, keeps one list's first place from outweighing everything else.

Q14 Why does the guard still use the embedding score?

The threshold, 0.50, was chosen on cosine scores, and BM25 scores have no fixed scale. Keeping the guard as it was means both searches refuse the same questions, so the evaluation measures only what hybrid search changes: the cited pages.

Q15 Why cite one chunk per page?

Several chunks of the same page repeat what the first one shows and take slots from other pages. Keeping each page's best chunk gives five different pages. It can't drop a page that was already cited, so the first citation stays and the right page can only be gained; it took the count from 24 to 26 of 30.

Q16 Why write BM25 yourself?

It's about 40 lines: no new library, and every line is tested. The `rank_bm25` package would work too; for a short formula, owning it was simpler.

 WHAT IF...?**Q17 What if the answer isn't in any of the papers?**

The best score is usually low, so the guard refuses and shows the score and the threshold. If a chunk is on topic but doesn't hold the answer, extractive mode will still show it, because it can't tell; mistral mode's second guard catches that case.

Q18 What if a PDF is scanned, a picture of text?

PyMuPDF finds no text, the PDF produces no chunks, and `build()` lists it as skipped. Fixing that needs OCR (reading text from images), which isn't built yet.

Q19 What if the answer is only in a table?

The noise filter drops blocks that are mostly numbers, so table contents are usually missing, and such a question will likely be refused. Table extraction is on the to-do list.

Q20 What if you had 10,000 papers?

That's roughly 300,000 to 500,000 chunks. Exact search still works but gets slower, so I'd switch to an approximate index (HNSW, or IVF-PQ to save memory), measure its accuracy against exact search, build the index in batches, and keep the text in a database.

Q21 What if two chunks disagree?

Extractive mode shows both with their pages, so you see the disagreement. Mistral mode is told to use only the passages and may mention both. Nothing resolves conflicts automatically.

 YOUR EVALUATION**Q22 How did you evaluate PaperRAG?**

With 51 questions about 8 papers: 34 answerable, each with its page and an evidence phrase, and 17 unanswerable, about models the papers never mention. A checker confirms every evidence phrase is on its page. Then a script counts unsupported answers, false refusals and how often the right page is cited, for every threshold from 0.20 to 0.66.

Q23 How did you choose the threshold?

From the sweep. Up to 0.42 nothing good is refused, but 8 of 17 unanswerable questions still get an answer. At 0.50 only 3 get through, for 4 wrongly refused good questions. Beyond that each extra catch costs one to three good questions, and 0.56 refuses a third of them.

Q24 "Unsupported answers fell from 33.3% to 5.9%": isn't that inflated?

A bit: without a guard every unanswerable question gets an answer, so the 33.3% is just their share of the set. I quote counts instead: the guard refused 14 of 17 unanswerable questions and wrongly refused 4 of 34 answerable ones.

Q25 How did you raise the right-page rate from 63% to 87%?

In two measured steps. The misses usually found the right paper but the wrong page, often for a question that hinges on an exact term, so I added BM25 keyword search and merged its ranking with the embeddings' by RRF: 19 → 24 of 30. Then the misses showed one page taking several of the five slots, so each page is cited once: 24 → 26, with nothing lost. Same 51 questions and the same guard each time.

Q26 Did hybrid search make anything worse?

One question lost its right page (one chunk per page later won it back), and the first citation was right 12 times instead of 13. Search takes about a millisecond more. The guard didn't change. On balance it's clearly better, but not on every number, and I report all of them.

Q27 Isn't the 87% flattered by how the questions were written?

Probably a little. The questions were drafted from the papers' passages, so they share words with the right page, which helps keyword search. Questions from real users would be the fairer test.

Q28 How were the questions written? Couldn't they be biased?

They were drafted from passages sampled across all 8 papers, reworded, and checked by hand. Because they start from the papers' own text, the citation score is probably a little higher than with real users' questions; the README says so.

Q29 Why did the first speed measurement show a p95 of 164 ms?

On a Mac the model runs on the GPU, which most likely does one-off setup work the first few times. Forcing the CPU instead crashed with a segmentation fault (two copies of the OpenMP threading library, most likely). Timing after one warm-up pass gave the real figure: p95 10.4 ms.

 WHAT MORE COULD YOU ADD?**Q30 What would you add next?**

1) A reranker over the top 20, because the first citation is right only 40% of the time and 4 right pages still rank below five others. 2) Testing mistral mode's second guard on the 3 unanswerable questions that slip through. 3) Citing only the passages the model actually used. 4) A stronger embedding model. 5) Tables and OCR.

Q31 How would a reranker help?

A cross-encoder reads the question and a passage together, so it judges the match better than comparing two separate vectors, but it's too slow to run on every chunk. So you rerank only the top 20 from hybrid search and put the best first, which is exactly the weak number: right page first, 40%.

Q32 How would you know that an improvement really helped?

Run the same 51 questions before and after and compare: that's how hybrid search (right page 19 → 24 of 30, first 13 → 12) and one chunk per page (24 → 26, none lost) were judged. If the numbers don't move, the change isn't worth its extra complexity.

 WHY NOT SOMETHING ELSE?**Q33 Why not paste all the papers into the LLM's prompt?**

They don't fit: dozens of papers are millions of characters. It would also be slow and expensive on every question, and models answer worse when the useful part is buried in a huge prompt.

Q34 Why not fine-tune a model on the papers?

Fine-tuning teaches style and behaviour, not reliable facts, and it can't point to a page. Adding a paper would mean training again; with RAG you just rebuild the index.

Q35 Why not use LangChain or LlamaIndex?

I wanted to understand and control every step, keep the dependencies few, and make each part easy to test. A framework would hide the chunking and the guard, which are the most interesting parts.

Q36 Why not a vector database like Pinecone?

At this size, a FAISS file is faster, free and simpler. A vector database pays off with many users, frequent updates, or filtering by metadata such as the user or the year.

 WHAT DID YOU TRY, AND WHAT DID YOU LEARN?**Q37 What went wrong with two-column papers, and how did you fix it?**

A test showed that blocks were sorted top to bottom, which mixed the columns. The simple fix of one sort key per block broke one-column pages, so I chose bands: wide blocks split the page, and each band is read left column first. Tests cover one-column, two-column and wide-caption pages.

Q38 What other bugs did the tests find?

The overlap carried into a new chunk wasn't counted, so chunks could reach about 1,050 characters, more than the model reads. And an overlap of 0 made every chunk repeat the whole previous one, because `text[-0:]` is the whole string. Both are fixed and tested.

Q39 Why do the tests use a fake embedder?

Unit tests should be fast, offline and give the same result every time. A word-counting stand-in keeps all three, while the real model's quality is measured separately by the evaluation.

Q40 What did adding hybrid search teach you?

To change one thing at a time: the guard stayed on the embedding score, so the comparison measured only the cited pages. And to read every miss, not just the total: 5 of the 6 left repeated a page in the citations, which pointed to the next cheap fix, one chunk per page: 24 → 26.

Q41 What did you learn from building it?

The hard parts of RAG aren't the LLM call: they're reading PDFs correctly, chunking, knowing when to refuse, and measuring. And tests find real bugs: two of the three chunking bugs appeared only because a test was written. Measuring then showed where the real weakness is: finding the right page, not the LLM.

PART 2 Text-to-SQL · sqlglot · read-only SQLite

AI

SchemaMind: Ask a Database in Plain English

IN THIS PART

- What problem SchemaMind solves, and the three ways text-to-SQL goes wrong
- The sample shop database, table by table
- **The whole pipeline in one picture**
- Each step: table cards, picking tables, writing SQL, checking it, running it, the repair loop
- **The safety layers**, and why safe is not the same as correct
- One question followed from start to finish, with real output
- **How well it works:** 24 questions, before and after two fixes
- What we changed in each session, and what we tried
- **Questions and answers**

2.1 The problem, in plain words

Companies keep their data in a **database**: tables of customers, orders and payments. To get an answer out of it you write **SQL**, a language for asking databases questions. Most people can't, so they wait for someone who can.

SchemaMind lets you type the question in English, *"How many orders came from Pune?"*. It writes the SQL, runs it **safely**, and shows you both the answer and the SQL.

Letting an LLM write SQL goes wrong in three ways, and SchemaMind has a mechanism for each:

What can go wrong	Example	SchemaMind's answer
Too many tables to describe	a real company database has hundreds of tables; describing them all in every prompt doesn't fit	retrieve the 3 most relevant table descriptions, plus the tables needed to join them
Destructive SQL	a confusing question produces <code>DROP TABLE customers</code>	a validator , a read-only connection and limits
Silent wrongness	valid SQL that runs fine but answers a different question	show the SQL; template mode refuses what it doesn't fully understand; the evaluation compares results

2.2 The sample shop database

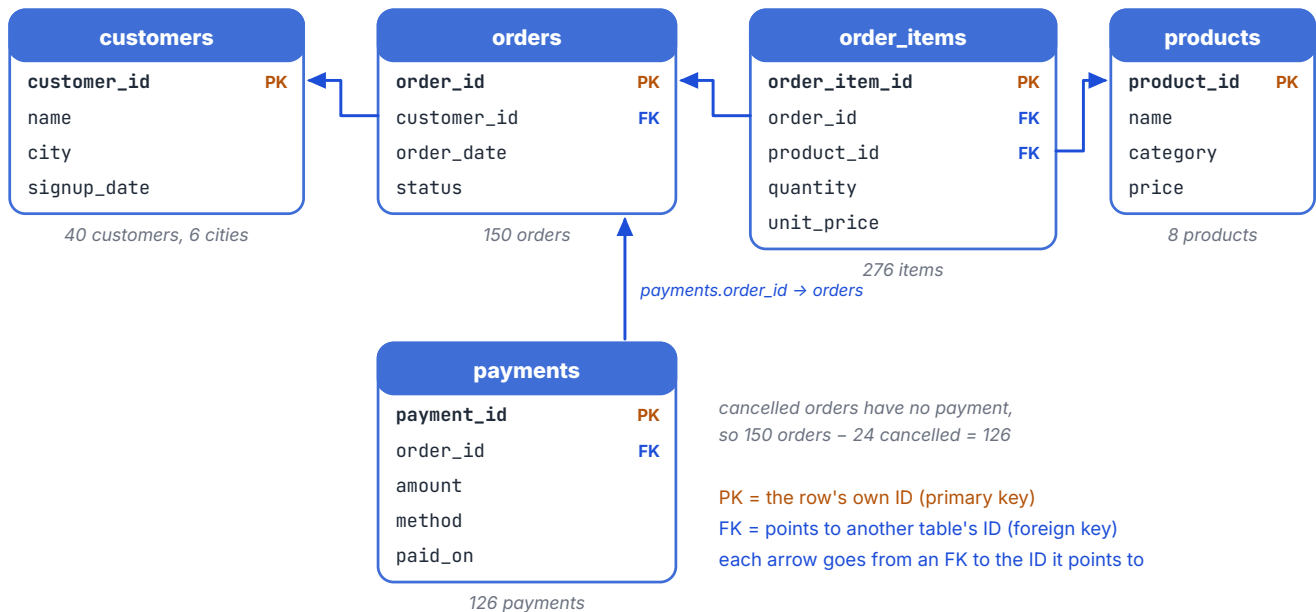


Figure 2.1 The five tables of `data/shop.db`. Each table has its own ID (PK); an FK column points to another table's ID, which is how the tables connect.

`python -m app.seed_db` builds this database with made-up data (always the same data, because it uses a fixed random seed). A few words you'll need:

- A **table** is a grid: each **row** is one thing (one customer, one order), each **column** is one fact about it.
- A **primary key (PK)** is a row's own ID, like `order_id`.
- A **foreign key (FK)** points to a row in another table. `orders.customer_id = 12` means "this order belongs to customer 12".
- A **JOIN** connects two tables through such a pair of IDs. The city lives in `customers` and the orders live in `orders`, so counting orders from Pune needs a JOIN.

2.3 The whole pipeline in one picture

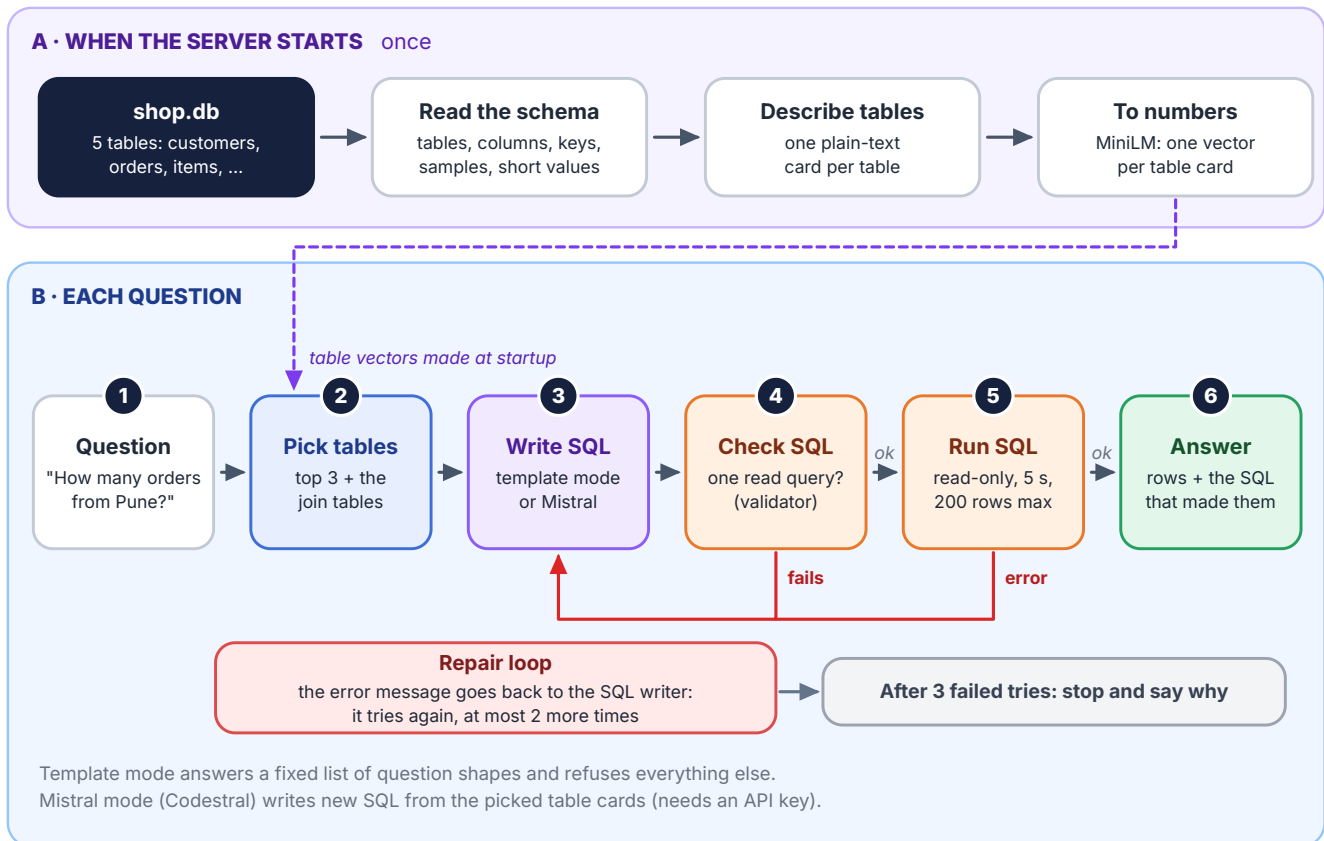


Figure 2.2 At startup SchemaMind writes a short card for every table and turns each card into numbers. For each question it picks the 3 most relevant tables plus the tables needed to join them, writes SQL, checks it, runs it safely, and returns the rows together with the SQL. Failures go back through the repair loop.

2.4 Each step, in order

A · Table cards, made once at startup

`introspect()` in `app/schema_introspect.py` reads the database's own description of itself: the table names, every column with its type, which columns are keys, two sample rows, and every value of a short text column. `describe()` turns each table into a short text card. This one is **real** (lines wrapped here):

OUTPUT the card for the orders table (real output, wrapped)

```

Table orders: order_id (INTEGER, PK),
  customer_id (INTEGER, -> customers.customer_id),
  order_date (DATE),
  status (TEXT, one of: 'cancelled', 'delivered', 'placed', 'shipped')
Sample rows from orders:
  order_id=1, customer_id=12, order_date=2025-09-07, status=placed
  order_id=2, customer_id=8, order_date=2026-06-09, status=delivered
  
```

The sample rows show what a row looks like: `status` holds words like `delivered`, not number codes. The **value list** (`one of: ...`) came in session 9, because the evaluation caught the model reading two sample rows as the complete list of values. The payments card showed only `card` and `cod`, so the model refused to total the UPI payments, though UPI is the most common method. Now a text column with at most 10 short values lists all of them: status, payment method, category, product name and city. Customer names (40 of them), numbers and dates are not listed.

Each card is then turned into numbers with MiniLM, the same model PaperRAG uses. Because the cards are read from the database itself, the project works on any SQLite database, not just this one.

B1-B2 · Pick the relevant tables

The question is turned into numbers too, and compared with every table card. The **3 most similar** cards are picked (`SchemaRetriever.relevant_tables()` in `app/schema_retriever.py`). Since session 9 it then adds the **join tables**, the ones needed to read and connect the picked tables:

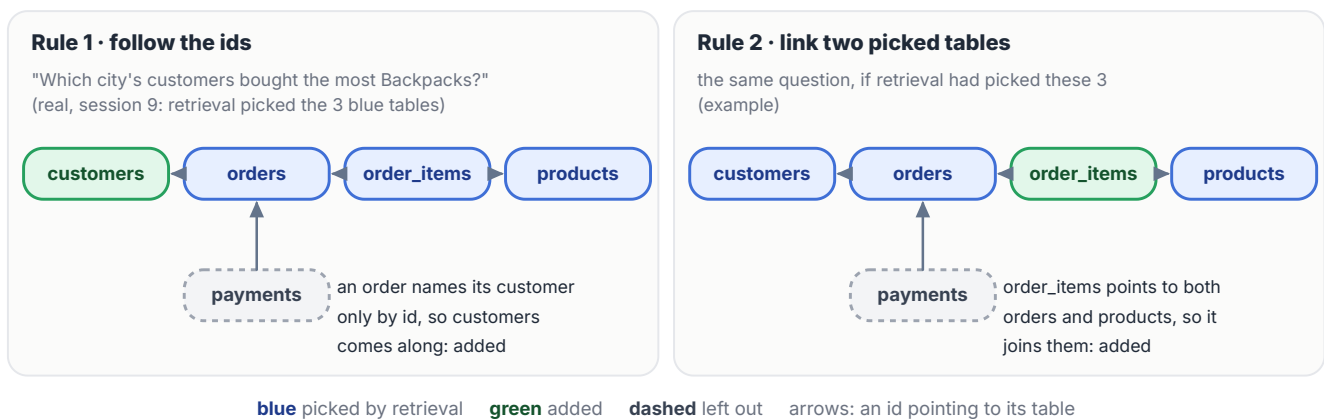


Figure 2.3 Rule 1: a picked table's foreign keys point to tables it can't be read without, so those come along. Rule 2: a table whose foreign keys link two picked tables is added, because the JOIN goes through it.

- Follow the ids.** Every table that a picked table's foreign keys point to. An `order_items` row names its product only as `product_id = 5`, which means nothing without `products`.
- Link two picked tables.** A table whose foreign keys point to two picked tables, like `order_items` between `orders` and `products`.

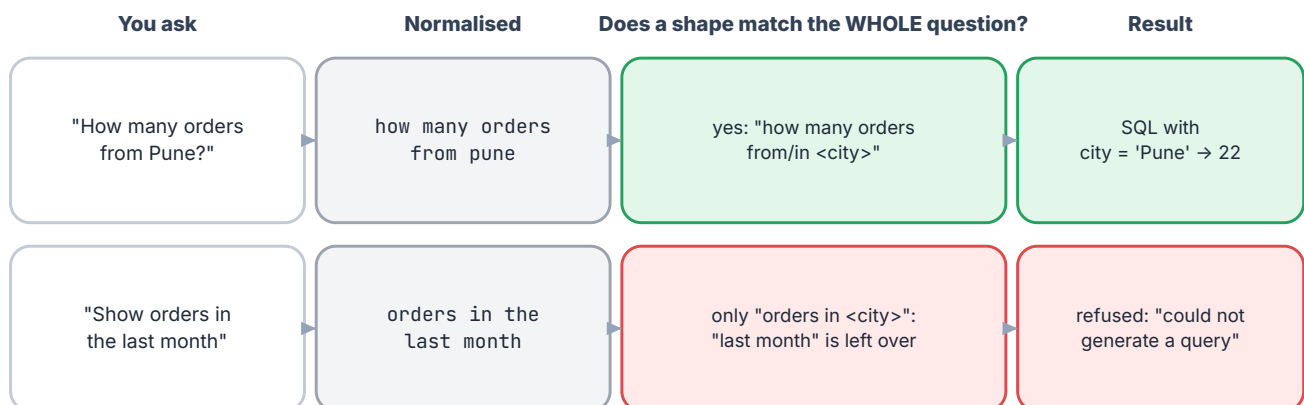
Before this, the prompt held every table a question needed for only 15 of the 20 evaluation questions, and when one was missing the model guessed instead of refusing (2.6). With the join tables it held them for all 20, with 4.2 tables in an average prompt instead of all 5. `SCHEMAMIND_JOIN_TABLES=0` turns the rule off.

With 5 tables this is barely needed. It is here because a real database has hundreds of tables, and you can only show a model a few of them. One honest detail: **template mode ignores the picked tables**, because its SQL is pre-written. The picked tables are shown in the reply, and they matter only in Mistral mode.

B3 · Write the SQL

There are two ways, set by `SCHEMAMIND_GEN_MODE`. **Template mode** (the default; no API key needed) knows a fixed list of question shapes:

Shape	Example	Real answer
how many <i>status</i> orders	how many cancelled orders	24
how many orders from/in <i>city</i>	how many orders from Pune	22
how many orders / customers / products	how many customers	40
top <i>N</i> customers by spend	top 3 customers by spend	Customer 7, 26, 15
top <i>N</i> customers by orders	top 3 customers by orders	9, 7, 7 orders
orders from/in <i>city</i>	orders from Indore	the 31 orders, listed
revenue by category / city	revenue by category	Electronics 4,703,200, Accessories 88,000
<i>status</i> orders	cancelled orders	the 24 orders, listed



lower case, "show me" / "what is the" / "?" removed; shapes are tried most specific first

Figure 2.4 Template mode first tidies the question, then needs a shape that matches the whole question. A question with words left over is refused instead of half-answered.

The rule that matters: the **whole** question has to match a shape. Matching only part of it is how template mode used to give wrong answers without any error (see 2.7).

Mistral mode is the real text-to-SQL. The picked table cards and the question go to the LLM with rules: write a single SQLite `SELECT`, use only the given tables and columns, never write `INSERT`, `UPDATE`, `DELETE`, `DROP`, `ALTER` or `CREATE`, return only the SQL, and reply `CANNOT_ANSWER` if the tables can't answer the question. Temperature is 0. It needs a `MISTRAL_API_KEY` in `.env`. The model is **Codestral** (`codestral-2508`), Mistral's model for code: your free plan refused every request to `mistral-small-latest`, and SQL is code anyway.

Since session 8 the client talks to Mistral carefully (`_post()` in `app/sql_generate.py`):

- Requests are spaced **1.1 seconds** apart, because the free plan limits how many you may send per second.
- A “too many requests” reply (**429**), a server error or a dropped connection is retried up to 4 times, waiting 1, 2, 4 and 8 seconds, or as long as Mistral’s `Retry-After` header asks.
- A missing or rejected key stops at once, and every error quotes **Mistral’s own reason**, such as *Rate limit exceeded*.
- The API turns a failed LLM call into a clear error (HTTP 502), and the web page shows its reason.

B4 · Check the SQL: the validator

The validator (`app/validate.py`) doesn’t search the text for bad words. It uses **sqlglot** to **parse** the SQL into a tree, the same way a compiler reads code, and checks the tree:

```
SELECT COUNT(*) FROM orders o JOIN customers c ON ... WHERE c.city = 'Pune'
```

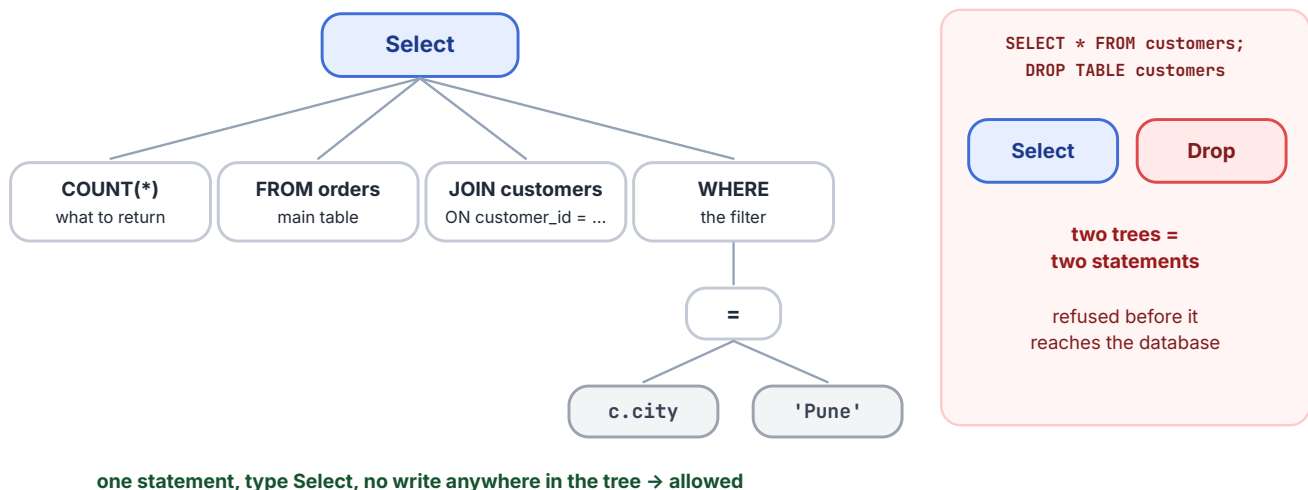


Figure 2.5 Left: how sqlglot sees a normal query. Right: two statements give two trees, which is refused.

1. **Exactly one statement.** `SELECT ...; DROP TABLE customers` is two statements, so it is refused.
2. **It must be a read query:** a `SELECT`, or several `SELECT`’s joined with `UNION`, `INTERSECT` or `EXCEPT`.
3. **No write hidden anywhere inside**, for example a `DELETE` tucked into a `WITH` part. It walks every node of the tree to check.

B5 · Run the SQL: the executor

`execute()` in `app/executor.py` runs the checked SQL with three protections:

- The database file is opened **read-only** (`mode=ro`). If a write ever got past the validator, SQLite itself refuses it: *attempt to write a readonly database*.
- A **5-second limit**. SQLite calls a small function every 10,000 steps of work; that function looks at the clock and, after 5 seconds, tells SQLite to stop. This stops queries that would run forever, like an endless recursive query or a join of four big tables.
- At most **200 rows** come back, so a huge result can’t flood the page.

Together with the validator, these make the safety layers:

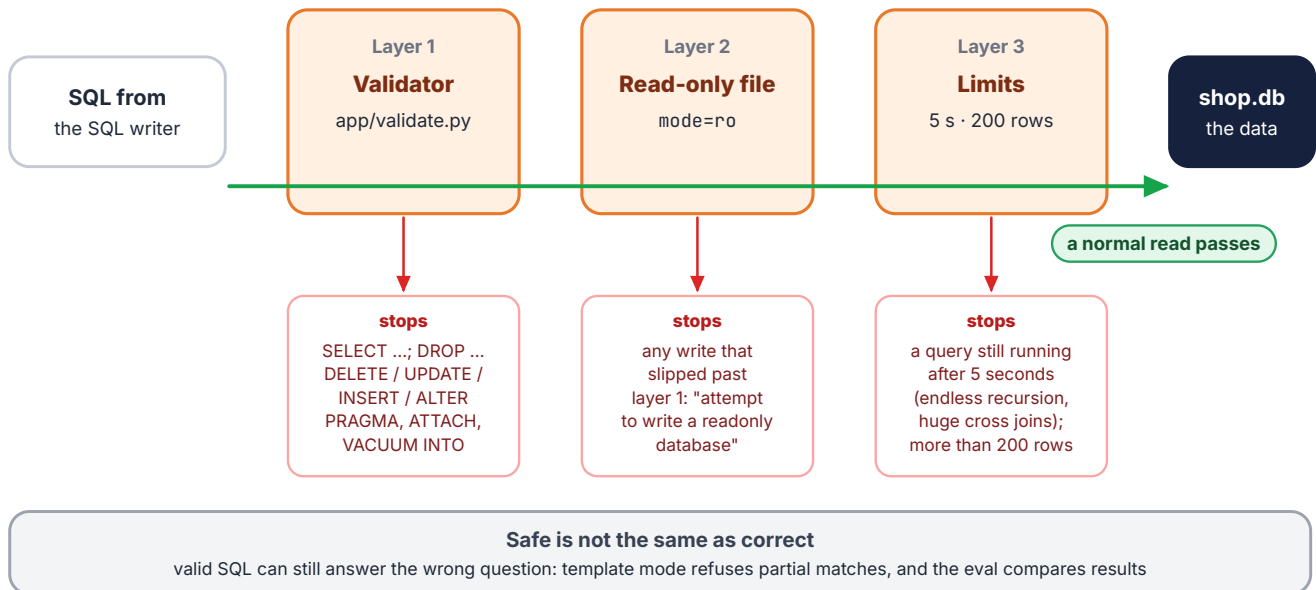


Figure 2.6 Each layer stops what the one before might miss. A normal read passes all three. None of them can tell whether a valid query answers the right question.

B6 · The repair loop

If the SQL fails the validator or the database returns an error, the error message and the failed SQL go back to the SQL writer with a note to fix it, and it tries again. There are at most **3 tries** (1 + `MAX_REPAIR_ATTEMPTS` = 2), then SchemaMind stops and reports the error. In Mistral mode the model can read an error like `no such column: city` and correct itself. In template mode the same SQL would come back, so a repair can't help there.

B7 · The reply

JSON what /ask sends back (real shape and values, template mode)

```
{
  "question": "How many orders from Pune?",
  "sql": "SELECT COUNT(*) AS count FROM orders o JOIN customers c ...",
  "ok": true, "error": null, "columns": ["count"], "rows": [[22]],
  "attempts": 1, "tables_used": ["orders", "customers", "payments"],
  "used_full_schema": false, "latency_ms": 12.4}

```

`tables_used` and `latency_ms` above are examples. Every answer comes **with its SQL**, so anyone who knows SQL can check how the number was produced.

2.5 One question, from start to finish (real output)

Question: “How many orders from Pune?”, in template mode.

Step	Result
Tidied question	<code>how many orders from pune</code>
Shape that matches the whole question	<i>how many orders from/in city</i>
SQL	the query below
Validator	one statement, a <code>Select</code> , no writes inside: OK
Executor	read-only, finished in milliseconds
Answer	<code>count = 22</code>

SQL the SQL that template mode wrote (real)

```
SELECT COUNT(*) AS count
FROM orders o JOIN customers c ON c.customer_id = o.customer_id
WHERE c.city = 'Pune';
```

Before session 5, the same question returned **150**, the total of all orders: “how many orders” matched first, and “from Pune” was ignored. The SQL was valid, so no safety layer noticed.

2.6 How it is measured and tested

Tests: 89, run with `pytest`, offline, in about ten seconds. They build their own copy of the database, never load MiniLM and never call Mistral: a fake Mistral server and a fake clock stand in. GitHub runs them after every push, on Python 3.11 and 3.14 (see 3.3).

Test file	What it checks	Tests
<code>test_validate.py</code>	read queries pass; a second statement, writes, <code>PRAGMA</code> , <code>ATTACH</code> , <code>VACUUM</code> are refused	23
<code>test_templates.py</code>	each shape gives the right answer; the five old wrong answers; partial matches refused; one definition of revenue	19
<code>test_mistral_client.py</code>	spacing, retries with growing waits, <code>Retry-After</code> , Mistral's reason in errors, a bad key stops at once	12
<code>test_eval.py</code>	how answers are scored; every gold query is safe, runs and returns rows; the early stop	11
<code>test_executor.py</code>	writes refused by the read-only file; runaway queries stopped at the time limit; row limit	8
<code>test_schema.py</code>	value lists for short columns only; the two join-table rules	8
<code>test_ui.py</code>	the page starts in the server's mode and shows why a request failed	3
<code>test_agent_safety.py</code>	end to end: an injected <code>DROP</code> is refused, the loop gives up, all 40 customers are still there	2
<code>test_config.py</code>	settings come from <code>.env</code> , and the shell wins	2
<code>test_api_errors.py</code>	a failed LLM call becomes a 502 that says why	1

Evaluation (`eval/run_eval.py`, sessions 8 and 9) measures how often the answers are right.

`eval/questions.jsonl` holds **24 questions** about the shop:

- **20 answerable**, each with **gold SQL**, the query you'd accept as correct. They go from one table (*How many customers are there?*) to four joined tables with two filters (*How many Laptops were bought by customers in Indore, in orders that were not cancelled?*).
- **4 unanswerable**: they ask for data the database doesn't hold (an email address, employees, profit, stock). The right answer is to refuse.

An answer counts when its **rows** match the gold rows: that is **execution accuracy**. The SQL text may differ, rows may come in any order, numbers are compared to 2 decimals, and extra columns are allowed (a customer's id next to the name the question asked for). Every question is asked twice, with the retrieved tables and with all 5 tables, and every miss is printed with its SQL.

`python -m eval.run_eval --mode template` is a free dry run of the whole script.

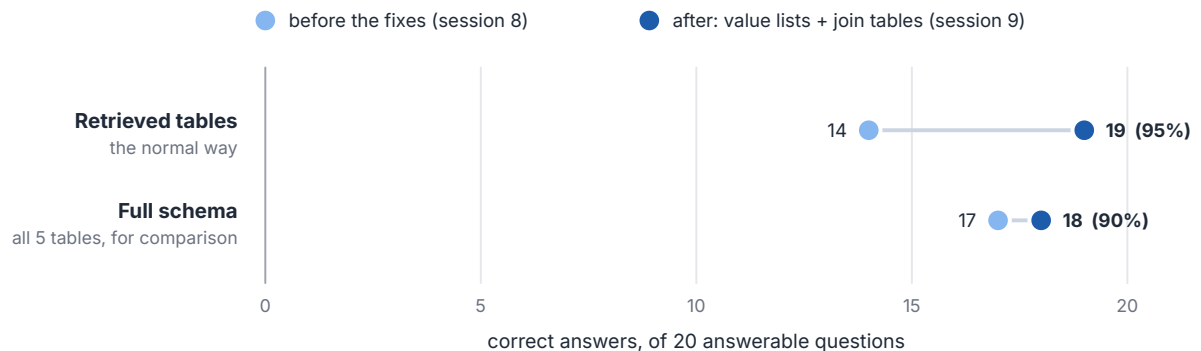


Figure 2.7 Your two runs (real, Codestral). With retrieval, 14 of 20 answers were right before the two fixes and 19 after; with the full schema, 17 and then 18.

Number (real)	Retrieved, before	Retrieved, after	Full, before	Full, after
Correct answers (of 20)	14	19	17	18
Ran, but returned wrong rows	3	1	0	2
Answerable, but refused	3	0	3	0
Unanswerable refused (of 4)	4	3	3	3
Every needed table in the prompt (of 20)	15	20	20	20
Tables in the prompt (average)	3	4.2	5	5

Before: two causes behind most misses. Every miss was checked against the database.

- *A missing table made the model guess.* For *How many delivered orders included a Monitor?* the query never touched `products` and filtered on `product_id = 5`, an id from the sample rows of `order_items`. Product 5 is the Mouse, so it answered 14 instead of 17, with no error: the **silent wrongness** from 2.1, measured.
- *Two sample rows hid the real values.* The payments card showed `card` and `cod`, so the model refused to total the UPI payments (69 of the 126). The Monitor and Backpack questions were refused the same way.

After: both fixes worked. Value lists removed every refusal of an answerable question, 3 to 0 in both modes, and join tables gave retrieval every needed table for all 20 questions. Retrieval now **matches** the full schema: 19 against 18 is one question, too small to call a win.

What still goes wrong. Three answers counted **order lines instead of items**: Indore has the most Backpack order lines (8), but Pune bought the most Backpacks (14 against 13). And in both modes the model **invented a profit** from the selling price minus the list price, though the shop records no costs.

2.7 What we changed, and what we tried

Session	Change	Why it matters
1	Every library fixed to a tested version	the same install everywhere
4	The validator checks every statement	a <code>DROP</code> hidden behind a <code>SELECT</code> was blocked only by luck, on newer sqlglot versions
4	A real 5-second limit	<code>timeout=10</code> only waited for locked files; endless queries ran forever
4	<code>INTERSECT</code> and <code>EXCEPT</code> allowed	they are reads but were refused; “customers who never ordered” is an <code>EXCEPT</code>
4	The first 33 tests	including the attacks above
5	Template mode needs the whole question to match	five valid-but-wrong answers fixed
5	One definition of revenue	“revenue by category” and “by city” now agree with total payments, 4,791,200
6	The tests run on GitHub after every push (CI)	a change that breaks something shows a red cross before it is merged
8	24 evaluation questions with gold SQL; real numbers in the README	Mistral mode had no accuracy numbers at all
8	Mistral requests spaced and retried; errors quote Mistral’s reason	the first run got a 429 on every request and never said why
8	Codestral is the default model	your free plan refused every request to <code>mistral-small-latest</code>
8	<code>.env</code> is read; the page starts in the server’s mode and shows why a request failed	a key put in <code>.env</code> was ignored, and errors showed only “502 Bad Gateway”
9	Short text columns list every value	the model refused the UPI, Monitor and Backpack questions
9	Join tables are added to the retrieved tables	without <code>products</code> the model guessed a product id

What we tried and found. The hidden second statement came from `parse_one()`, which in sqlglot 26–28 looked only at the first statement; the new check was tested on sqlglot 26, 27, 28, 29 and 30. The time limit was found by writing a recursive query that never ends and watching it keep running. The five wrong template answers were found by asking natural questions and comparing with SQL written by hand: *how many orders from Pune* (150, not 22), *Revenue by Category* (grouped by city), *orders in the last month* (searched for a city called “The”), *top 3 customers by orders* (ranked by money), *how many cancelled orders* (listed rows instead of a count).

What we tried in sessions 8 and 9. The first real run got a 429, *too many requests*, on every single request, even with 1.1 seconds between them and waits of up to 8 seconds, and spent 14 minutes failing all 48. That couldn’t be the per-second limit, so the client was changed to quote Mistral’s reason

and the evaluation to stop after 3 questions in a row get no answer. The reason, *Rate limit exceeded*, pointed at the account: the Limits page of Mistral's console listed `codestral-2508` with 2 requests per second, and switching to it worked at once. The two fixes of session 9 came straight from reading every miss of the first full run.

Still missing

Three answers count order lines where the question asks for items, and the model invents a profit the data can't give: telling it what `quantity` means and that the shop records no costs is the next step. 24 questions is a small set. No per-user permissions, and no clarifying questions for ambiguous words like "best".

2.8 Questions and answers

HOW DOES IT WORK?

Q1 How does SchemaMind know which tables a question needs?

At startup it writes a short card for every table (columns, keys, two sample rows, the values of short text columns) and turns each card into numbers with MiniLM. The question is turned into numbers the same way, and the 3 most similar cards are picked. Then the join tables are added: the tables the picked ones' foreign keys point to, and any table that links two of them.

Q2 How does it stop a `DROP TABLE` ?

The validator parses the SQL into a tree and refuses anything that isn't a single read query. If something ever got past it, the database file is open read-only, so SQLite refuses any write. The prompt also forbids writes, but that's a request, not a control.

Q3 How does it stop a query that would run forever?

SQLite calls a small function every 10,000 steps of work. The function checks the clock and, after 5 seconds, tells SQLite to stop. The query ends with "interrupted", which SchemaMind turns into a clear message.

Q4 How does the repair loop work?

If the SQL fails a check or the database returns an error, the error text and the failed SQL go back to the SQL writer to fix. It gets up to two more tries; after three failures it stops and reports the error.

Q5 How does template mode turn a question into SQL?

It lower-cases the question and removes openings like "show me" and the question mark. Then it tries a fixed list of patterns, most specific first. A pattern must match the whole question; its blanks (a city, a status, a number) are filled into ready-made SQL. If nothing matches completely, it refuses.

Q6 How does a JOIN answer “orders from Pune”?

The city is stored in `customers` and the orders in `orders`. Join them on `customer_id`, keep the rows where the city is Pune, and count them: 22.

**WHY WAS IT BUILT THIS WAY?****Q7 Why retrieve tables instead of sending the whole schema?**

With 5 tables you could send everything. A real company has hundreds of tables, which don't fit in a prompt and would confuse the model. Picking the relevant ones is the technique the project shows; the evaluation is built to measure whether it helps.

Q8 Why list column values instead of showing more sample rows?

More rows would still miss values: UPI is the most common payment method, and it wasn't in the first two rows. Listing the distinct values covers all of them for short columns. Columns with many values, like 40 customer names, aren't listed, so the cards stay short.

Q9 Why add join tables instead of retrieving more tables?

A bigger top-k adds tables that look similar to the question, not the ones it needs. The join tables are exactly what the picked tables can't be read or joined without, found from the foreign keys. Here that meant 4.2 tables per prompt, not all 5.

Q10 Why Codestral?

Your free plan refused every request to `mistral-small-latest`, and Codestral is Mistral's model for code. SQL is code, and the evaluation numbers come from it, so it's the default; `MISTRAL_MODEL` in `.env` changes it.

Q11 Why parse the SQL instead of searching for words like DROP?

Word searches are easy to fool and easy to get wrong: a column named `dropped_at` would be blocked, while tricks with comments or spacing might slip through. A parser understands the structure, so it can count statements and check their type reliably.

Q12 Why have both a validator and a read-only connection?

Because any single check can have a bug, and ours did (the hidden second statement). With two independent layers, a mistake in one is caught by the other. The read-only file is enforced by SQLite itself, so no trick against the parser can make it write.

Q13 Why show the SQL with every answer?

A wrong JOIN gives a believable number, not an error. Showing the SQL lets a person check how the answer was produced.

Q14 Why a 5-second limit and only 200 rows?

On this database a normal query takes milliseconds, so anything past 5 seconds is a runaway. Answers are read by people, so 200 rows is plenty. Both are settings that a bigger database could raise.

Q15 Why one definition of revenue?

Two templates computed "revenue" from different tables and disagreed by 631,600 (5,422,800 vs 4,791,200), because one counted cancelled orders. Defining revenue once, as items in orders that weren't cancelled, makes every answer consistent. Real companies call such shared definitions a **semantic layer**.

 WHAT IF...?**Q16 What if the LLM writes valid SQL that answers the wrong question?**

The safety layers won't notice, because nothing unsafe happened. The evaluation caught exactly this: with **products** missing from the prompt, the model filtered on a guessed product id and returned 14 instead of 17, with no error. That's why the SQL is shown, why the evaluation compares results against gold SQL, and why template mode refuses questions it only partly understands.

Q17 What if the database had 500 tables?

That's what retrieval is for: only the top few cards go to the model, plus the join tables found from the foreign keys. I'd keep measuring whether the prompt holds every table the gold SQL uses (the evaluation counts it: 20 of 20 here), write good table descriptions, and keep value lists to short columns, as now.

Q18 What if a user asks for data they shouldn't see, like salaries?

Today every user can read every table. In a real company I'd connect with a database user that may read only allowed tables or columns, filter rows per user (row-level security), and hide personal data.

Q19 What if Mistral is down or slow?

Each request is retried up to 4 times, waiting 1, 2, 4 and 8 seconds, or as long as Mistral's **Retry-After** asks. If it still fails, the API returns a 502 with Mistral's reason, which the page shows, and the evaluation stops after 3 questions in a row get no answer. Falling back to template mode for the shapes it knows would be a next step.

Q20 What if the question is ambiguous, like "best customers"?

"Best" could mean most money or most orders. Template mode knows only "by spend" and "by orders", so it refuses plain "best". An LLM would guess; a better system would ask which one you mean.

Q21 What if someone types SQL instead of a question?

In template mode it matches no shape and is refused. In Mistral mode the model might copy it, but it still has to pass the validator and run on the read-only file.

 YOUR EVALUATION**Q22 How did you evaluate SchemaMind?**

With 24 questions about the shop: 20 with gold SQL, from one table to four joined tables, and 4 the database can't answer. An answer counts when its rows match the gold rows, in any order and with extra columns allowed: that's execution accuracy. Every question runs twice, with the retrieved tables and with all 5, and every miss is printed with its SQL.

Q23 What were the results?

With retrieval, 14 of 20 right at first and 19 of 20 after two fixes; with the full schema, 17 and then 18. Refusals of answerable questions went from 3 to 0, and the prompt held every needed table for 20 of 20 questions, up from 15.

Q24 How did you find the two fixes?

By checking every miss against the database. One query filtered on `product_id = 5` because `products` wasn't in the prompt, and product 5 is the Mouse, not the Monitor. And the model refused to total UPI payments because the two sample rows it saw showed only card and cod. So: add the tables needed for joins, and list the values of short columns.

Q25 Retrieval scored 19 and the full schema 18. Is retrieval better?

No. One question out of 20 is within noise. The honest claim is that retrieval now matches the full schema while sending 4.2 tables instead of 5, and on a warehouse with hundreds of tables the full schema wouldn't fit at all.

Q26 What still goes wrong?

Three answers count order lines where the question asks for items: Indore has the most Backpack order lines, but Pune bought the most Backpacks. And the model invents a "profit" from list prices, though the shop records no costs. Both point to the same fix: tell the model what the data means.

Q27 What happened with the Mistral API?

Every request got a 429, "too many requests", even the first and even after waiting, and the client didn't say why. So I made it quote Mistral's reason and stop after three failed questions. The reason was a limit on the account: the model wasn't usable on the free plan. The console's Limits page listed Codestral, and switching to it worked at once.

 WHAT MORE COULD YOU ADD?**Q28 What would you add next?**

1) Tell the model what `quantity` means and that the shop records no costs, the two remaining mistakes. 2) More evaluation questions: 24 is small. 3) A glossary of business words ("revenue", "active customer") given to the model. 4) Clarifying questions. 5) Per-user permissions.

Q29 How would you make the evaluation stronger?

More questions, ideally written by someone else, and harder ones: more joins, dates, ties. Run each a few times, because even at temperature 0 an answer can change when the prompt changes. And keep some questions aside that no fix was tuned on, so a better score means a better system, not one fitted to the test.

Q30 What is self-consistency, and would it help?

Generate several SQL queries for the same question, run them all, and keep the answer most of them agree on. It catches some one-off mistakes, at the cost of more LLM calls.

**WHY NOT SOMETHING ELSE?****Q31 Why not a list of banned words?**

It blocks harmless queries and misses tricky ones. A parser works on structure, so it can count statements and check their type reliably.

Q32 Why not let the LLM run SQL on a normal connection and trust the prompt?

A prompt is a request, not a control: the model can misunderstand, or be tricked by the question. Safety has to be enforced by code and by the database itself.

Q33 Why not a longer limit, like 60 seconds?

Every running query ties up the server, and minute-long runaways would make the app feel broken. 5 seconds is generous for this database, and it's a setting (`SCHEMAMIND_QUERY_TIMEOUT`) if a bigger database needs more.

Q34 Why not LangChain's SQL agent?

It would work, but it hides the parts I wanted to build and explain: table retrieval, the validator, the limits and the repair loop. Building them myself means I can test each one.

**WHAT DID YOU TRY, AND WHAT DID YOU LEARN?****Q35 Tell me about the hidden second statement.**

The validator used `parse_one()`, which in older sqlglot versions read only the first statement, so a `DROP` after a `SELECT` went unchecked. It was safe on newer versions only by luck. I changed it to parse every statement and allow exactly one, and tested it on sqlglot 26 to 30.

Q36 How did you find that the timeout didn't work?

I wrote a recursive query that never ends and ran it: it kept running. `timeout=10` only limits waiting for a locked file. I added SQLite's progress handler with a deadline, and a test proves that such a query stops at the limit.

Q37 What were the five wrong template answers?

Template mode took the first pattern that matched part of a question. "How many orders from Pune" gave 150 instead of 22, "Revenue by Category" grouped by city, "orders in the last month" searched for a city called "The", "top 3 customers by orders" ranked by money, and "how many cancelled orders" listed rows. Now the whole question must match, specific shapes come first, and everything else is refused.

Q38 What was wrong with INTERSECT and EXCEPT?

They are read queries ("customers who never ordered" is all customers EXCEPT those with orders), but the validator refused them because it accepted only `SELECT` and `UNION`. Now any combination of `SELECT` s is allowed.

Q39 What did the evaluation teach you?

That measuring finds causes you wouldn't guess. I expected retrieval to lose a little on a small database; I didn't expect the model to guess a product id from sample rows, or to read two sample rows as the full list of values. Each finding became a fix, and the same 24 questions showed it worked.

Q40 What did you learn?

That safe and correct are different problems: three safety layers can't stop a believable wrong number. And that every claim in a README needs a test that tries to break it: the injection check worked only by luck, and template mode, marked "tested, works", gave wrong answers to five natural questions.

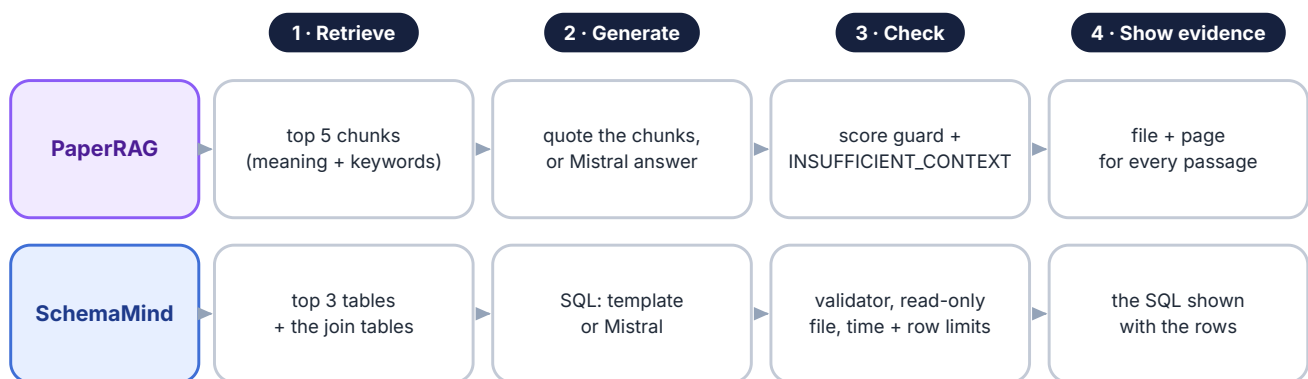
Seeing Them Together

IN THIS PART

- **The same four steps** inside both projects
- PaperRAG and SchemaMind side by side
- How to explain each project in one minute
- Questions that cover both projects
- **Glossary** of every word in this guide

3.1 The same skeleton

Put the two pipelines next to each other and the same four steps appear. Once you see them, you can explain either project in the same order, and you already have an answer to “What do your projects have in common?”.



Both refuse honestly: PaperRAG says "I don't know", SchemaMind says it can't write a query.

Figure 3.1 Both projects first find the right material, then make an answer from it, check it, and show the evidence.

What each one does

	PaperRAG	SchemaMind
You ask about	a folder of research papers	the database of a small shop
What is searched	text chunks of up to 900 characters	one card per table
How it searches	meaning (MiniLM, FAISS) + keywords (BM25), merged; top 5 pages, one chunk each	MiniLM numbers, top 3 + the join tables
The answer is made of	the passages, or Mistral's answer written from them	an SQL query and the rows it returns

How each one stays honest

	PaperRAG	SchemaMind
The main check	is the best score at least 0.50?	one read query only, then read-only, 5 s, 200 rows
The evidence shown	file and page of every passage	the SQL itself
Its honest “no”	“I don’t have enough supporting material...”	“Could not generate a query for this question.”
Without an API key	extractive mode	template mode
Tests, run by CI on every push	70	89
Biggest open item	the first citation is right only 40% of the time	3 answers count order lines instead of items

Two differences are worth noticing. PaperRAG’s worst mistake is a **wrong answer**; SchemaMind’s output is **code that runs on a database**, so its worst mistake could be a deleted table. That’s why SchemaMind has three safety layers where PaperRAG has one guard. And SchemaMind doesn’t need FAISS: comparing a question with 5 table cards takes one line of numpy.

3.2 Explaining each project in one minute

Every project story has the same five parts. Here they are for both projects, so you can see the structure before you read the full versions.

Part	PaperRAG	SchemaMind
The problem	LLMs make things up and don’t say where an answer came from	most people can’t write SQL, and SQL written by an LLM can be unsafe or quietly wrong
How it works	blocks → chunks → meaning + keywords → top 5 → guard → answer	table cards → top 3 + join tables → SQL → validator → read-only run
A decision you can defend	a chunk never crosses a page, so every chunk has one page number	the SQL is parsed into a tree instead of being searched for bad words
How you know it works	70 tests; on 51 questions the guard refused 14 of 17 unanswerable ones and 4 of 34 good ones, and the right page is cited for 87% (from 63%)	89 tests, including real attacks; on 24 questions, 19 of 20 right with retrieval (from 14)
What comes next	a reranker for the first citation; a stronger embedding model	tell the model what quantity means; more questions; per-user permissions

PaperRAG in one minute

"PaperRAG answers questions about a folder of research papers and gives the **file and page** of every passage it used. It reads each PDF in visual text blocks, puts two-column pages in reading order, and packs the blocks into chunks of at most 900 characters that **never cross a page**, so every chunk carries exactly one page number. Each chunk becomes a 384-number MiniLM vector in a FAISS index. For a question it ranks the chunks by meaning and by keywords with BM25, merges the two rankings and cites the top five pages; if even the best embedding score is below a threshold, it **refuses instead of guessing**. By default it returns the passages word for word, so it can't invent anything; with an API key, Mistral writes a short answer from them, with a second refusal check. It has a FastAPI server, a Streamlit page and 70 offline tests. On 51 hand-checked questions about 8 papers, the guard refused 14 of the 17 unanswerable questions while wrongly refusing 4 of the 34 answerable ones, I picked the threshold, 0.50, from that sweep, and keyword search plus citing each page once raised the share of answers citing the right page from 63% to 87%."

SchemaMind in one minute

"SchemaMind lets you ask a database a question in plain English and shows the answer **together with the SQL** that produced it. It reads the database's own structure and writes a short card for every table, with columns, keys, two sample rows and the values of short columns. For a question it picks the three most relevant cards with MiniLM and adds the tables needed to join them, so the prompt stays small even when a database has hundreds of tables. The SQL comes from Mistral's Codestral, or from a template mode that needs no API key and **refuses any question it doesn't match completely**. Every query then passes **three safety layers**: sqlglot parses it and allows exactly one read statement, the database file is opened read-only, and a five-second limit and a 200-row cap stop runaway queries. Errors go back to the SQL writer for at most two repairs. 89 offline tests include real attacks, such as a **DROP** hidden behind a **SELECT**. On 24 hand-written questions it answered 14 of 20 correctly at first; reading every miss showed two causes, and fixing them took it to 19 of 20."

How to practise the pitches

Say each one out loud with a timer: aim for 60–70 seconds. Don't learn the words by heart; learn the five parts in the table above and say them your own way. Then stop and let the interviewer pick what to dig into. The question sections in Parts 1 and 2 are where they will dig.

3.3 Questions that cover both projects

Q1 What do your two projects have in common?

Both find the right material first and answer only from it. Both check before answering, show their evidence (pages or SQL), and refuse instead of guessing. Both also run without an API key, which made them easy to test offline.

Q2 Both use MiniLM. Why the same model?

It is small (about 80 MB), runs on a normal laptop CPU in milliseconds, and is good enough for matching short texts. In SchemaMind it only has to pick 3 of 5 table cards, so even a weak model works. In PaperRAG it matters more, and the evaluation is where you would test whether a bigger model is worth it.

Q3 PaperRAG uses FAISS and SchemaMind doesn't. Why?

SchemaMind compares the question with 5 cards, which numpy does in one step. PaperRAG may hold thousands of chunks and saves them to disk, so a real vector index is worth it. Use the simplest tool that fits the size of the problem.

Q4 Which project is harder to evaluate?

PaperRAG. You need questions, the pages that answer them, and a judgement of whether each answer is supported. SchemaMind has an exact referee: run the gold SQL and compare the rows.

Q5 How are the two "I don't know"s different?

PaperRAG refuses when its search finds nothing close enough, or when Mistral says the passages don't contain the answer. SchemaMind refuses when it can't write a query: no template matches the whole question, or Mistral replies `CANNOT_ANSWER`. Both prefer a clear "no" to a believable wrong answer.

Q6 Could you combine the two projects?

Yes, as one assistant with two tools: questions about documents go to PaperRAG's search, questions about numbers go to SchemaMind. A router, either simple rules or an LLM, picks the tool, and each tool keeps its own checks. Letting a model choose its tools like this is what **agentic RAG** means.

Q7 How do you know a change didn't break anything?

Both projects have offline tests, 70 and 89, that finish in seconds. **GitHub Actions** runs all of them after every push, on a fresh Linux machine, with Python 3.11 and with 3.14. A pull request shows a green tick or a red cross before it is merged, and the badge in each README shows the state of `main`.

Q8 Why does CI install the CPU-only build of PyTorch?

PyTorch comes in with sentence-transformers, and its default Linux build includes several GB of GPU libraries. The tests never run a model and GitHub's machines have no GPU, so the much smaller CPU-only build is enough. A whole run takes about a minute and a half.

Q9 Why test on Python 3.11 and 3.14?

3.11 is the oldest version the READMEs promise, and 3.14 is the one on your Mac. Testing both ends catches code, or a pinned library, that works on one version but not on the other.

Q10 What do your two evaluations have in common?

Both run the same questions before and after a change, and both read every miss instead of trusting the total. And both fixes gave the model exact words: BM25 matches the question's rare terms in PaperRAG, and value lists show a column's real values in SchemaMind.

Q11 What would you do with one more week?

Fix what each evaluation found: in PaperRAG, a reranker for the first citation; in SchemaMind, tell the model what `quantity` means and that the shop records no costs. Then re-run the same questions. CI already runs all 159 tests on every push, so each change would be checked automatically.

Q12 What did building both projects teach you?

Testing finds bugs that reading the code doesn't: two chunking bugs, an injection check that worked only by luck, a timeout that never fired and five wrong template answers. Measuring finds the real weakness: the right page in PaperRAG, missing tables and hidden values in SchemaMind. And an honest refusal beats a confident wrong answer, so both projects are built to say no.

3.4 Glossary

Every word in this guide, in one line each. The last column says which project uses it.

Word	What it means	Used in
Agentic RAG	an LLM decides which search or tool to use for each question	idea
API	a way for one program to ask another for something; here, over HTTP	both
Band	a strip of a two-column page between two wide blocks	PaperRAG
BM25	keyword search: scores a passage by the question's words it contains, rare words most	PaperRAG
Block	a paragraph-sized piece of text on a PDF page, with its position	PaperRAG
Chunk	up to 900 characters of text from one page; the piece PaperRAG searches	PaperRAG
CI	continuous integration: GitHub runs the tests on every push	both
Citation	the file and page that a passage came from	PaperRAG
Citation hit rate	how often the right page is among the cited ones (87% in PaperRAG)	PaperRAG
Codestral	Mistral's model for code; SchemaMind's SQL writer (<code>codestral-2508</code>)	SchemaMind
Cosine similarity	how closely two vectors point the same way; 1 = same meaning	both
Dot product	multiply two lists number by number, then add; for vectors normalised to length 1 it equals cosine similarity	both
Embedding	a vector: a list of numbers (384 here) that describes what a text means	both
Endpoint	an address on a server that does one job, like <code>/ask</code> or <code>/health</code>	both

Word	What it means	Used in
Evidence phrase	a short phrase copied from the page that answers a test question	PaperRAG
Execution accuracy	how often SchemaMind's rows equal the rows of the gold SQL (19 of 20 with retrieval)	SchemaMind
Extractive mode	PaperRAG's default: return the passages word for word	PaperRAG
FAISS	a library that finds the closest vectors quickly	PaperRAG
False refusal	a question the system could answer but refused	PaperRAG
FastAPI	the Python library both web servers are built with	both
Foreign key (FK)	a column that points to a row in another table	SchemaMind
Gold SQL	SQL you wrote by hand as the correct answer, used for grading	SchemaMind
Guard	refuse when the best score is below the threshold	PaperRAG
Hallucination	an LLM stating something false in a confident voice	both
Hybrid search	search by meaning and by keywords together, with the rankings merged	PaperRAG
Index	a structure built for fast searching; PaperRAG's is a FAISS index	PaperRAG
JOIN	connect the rows of two tables through matching IDs	SchemaMind
Join tables	tables added to the retrieved ones so they can be read and joined	SchemaMind
JSON	a simple text format for data, like <code>{"question": "..."} </code>	both
LLM	large language model, such as Mistral or GPT	both
MiniLM	all-MiniLM-L6-v2, the small embedding model both projects use	both
Mistral mode	the LLM writes the answer (PaperRAG) or the SQL (SchemaMind)	both
Overlap	the end of one chunk repeated at the start of the next, 150 characters	PaperRAG
Parse	read text into a structure (a tree) by following grammar rules	SchemaMind
Primary key (PK)	a row's own ID, like <code>order_id</code>	SchemaMind
Progress handler	a function SQLite calls every 10,000 steps; used for the time limit	SchemaMind
PyMuPDF	the library that reads text and positions out of PDFs	PaperRAG
pytest	the tool that finds and runs the tests (code that checks other code)	both
RAG	retrieval-augmented generation: find relevant text first, then answer from it	PaperRAG
Rate limit (429)	a cap on requests per second; the API answers 429, "too many requests"	SchemaMind
Read-only	opened so that nothing can be changed (<code>mode=ro</code>)	SchemaMind

Word	What it means	Used in
Reciprocal rank fusion (RRF)	merge rankings: each gives an item $1 / (60 + \text{rank})$, and the shares add up	PaperRAG
Regex	a pattern for matching text; template mode's shapes are regexes	SchemaMind
Repair loop	send the error back to the SQL writer and try again; 3 tries at most	SchemaMind
Retrieval	finding the pieces most relevant to a question	both
Schema	the structure of a database: its tables, columns and keys	SchemaMind
Semantic layer	one shared definition of business words like "revenue"	SchemaMind
sqlglot	a Python library that parses SQL into a tree	SchemaMind
SQLite	a small database stored in a single file, here <code>shop.db</code>	SchemaMind
Streamlit	a library for quick web pages in Python; both projects' UI	both
Sweep	running the evaluation once for every threshold, to pick one	PaperRAG
Table card	SchemaMind's short description of one table, with two sample rows and short value lists	SchemaMind
Temperature	how random an LLM's output is; 0 gives the same answer every time	both
Template mode	SchemaMind's default: fixed question shapes with ready-made SQL	SchemaMind
Threshold	the cut-off score; PaperRAG uses 0.50, picked from its evaluation	PaperRAG
Top-k	the k best matches: 5 chunks, or 3 tables before the join tables are added	both
Validator	SchemaMind's SQL checker (<code>app/validate.py</code>)	SchemaMind
Value list	every value of a short text column, shown in its table card	SchemaMind
Word piece	the small text pieces a model reads; MiniLM reads up to 256	both